

OpenMP Exam Prep

Every Likely Code Pattern — Question, Answer & Simple Step-by-Step Explanation

The Golden Rule

OpenMP questions are 100% confirmed to appear on the test. The exact logic may change (e.g. factorial instead of sum, product instead of max), but the **structure never changes**. If you master how to place the `#pragma omp . . .` directive line, you can adapt to any logic thrown at you. There are really only two paradigms:

- **Parallel computation** — split a loop's iterations across threads (`#pragma omp parallel for`).
- **Shared memory** — multiple threads read/write the same variable safely (reduction, critical, atomic).

How to use this guide

Read every question below once. Don't memorize each program word for word — instead notice the ONE line that changes each time (the pragma line and the loop body). That line is the whole trick. Everything else (includes, main, printf, return 0) is copy-paste boilerplate you already know.

What's inside

1. Hello World — basic parallel region
2. Print thread ID (private clause)
3. Parallel for loop (splitting iterations)
4. Array sum using reduction
5. Array product using reduction
6. Factorial of a number using reduction
7. Find maximum in array using reduction
8. Find minimum in array using reduction
9. Count even/odd numbers using reduction
10. Shared variable WITHOUT protection (race condition demo)
11. critical section (safe shared update)
12. atomic (lightweight safe update)
13. sections — different tasks on different threads
14. private vs shared clause comparison
15. Setting number of threads manually

16. Matrix (2D array) addition
17. Sum of squares 1..N
18. Linear search in parallel (with critical)

Q1. Write a simple OpenMP program that prints a message from multiple threads.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel
    {
        printf("Hello from thread\n");
    }
    return 0;
}
```

Sample Output

```
Hello from thread
Hello from thread
Hello from thread
Hello from thread
```

Explanation — Step by Step (Simple Words)

1. Include `<omp.h>` — this gives you all OpenMP functions and pragmas.
2. The line `#pragma omp parallel` tells the compiler: 'run the block right after this in parallel, on multiple threads'.
3. The `{ }` block after the pragma is called the **parallel region**. Every thread runs a copy of everything inside it.
4. If your CPU has 4 cores, OpenMP usually creates 4 threads, so the `printf` line runs 4 times.
5. The order of the lines in the output is NOT guaranteed — threads can finish in any order.

Note: This is the base template for every OpenMP question. Everything else is built on top of it.

Q2. Modify the program so each thread prints its own thread ID.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int id;
    #pragma omp parallel private(id)
    {
        id = omp_get_thread_num();
        printf("Hello from thread %d\n", id);
    }
    return 0;
}
```

Sample Output

```
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3
```

Explanation — Step by Step (Simple Words)

1. `omp_get_thread_num( )` is a built-in function that returns the ID of the thread currently running (0, 1, 2, 3, ...).
2. `private(id)` means each thread gets its OWN separate copy of the variable `id` — they don't overwrite each other's value.
3. Without `private(id)`, all threads would share one `id` variable and the numbers printed could get mixed up or repeated.
4. Thread 0 is always the 'master' thread (the one `main()` started with); the rest (1,2,3...) are the extra worker threads OpenMP creates.

Note: Any time a variable must hold a DIFFERENT value per thread, make it private.

Q3. Parallelize a simple for loop so its iterations run on different threads.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel for
    for (int i = 0; i < 10; i++) {
        printf("i = %d\n", i);
    }
    return 0;
}
```

Sample Output

```
i = 0
i = 4
i = 1
i = 5
i = 2
i = 6
i = 3
(order may vary each run)
```

Explanation — Step by Step (Simple Words)

1. `#pragma omp parallel for` is TWO things joined together: (a) create a team of threads, (b) automatically split the loop's iterations among them.
2. Example: with 10 iterations and 4 threads, roughly thread 0 gets `i=0,1,2` ; thread 1 gets `i=3,4,5` ; and so on (the exact split is decided by OpenMP's default 'static' scheduling).
3. The loop variable `i` is automatically made private by OpenMP for you — you do NOT need to write `private(i)` yourself for the loop counter.
4. Because iterations run at the same time on different cores, the printed order is scrambled — that's expected and fine.

Note: This is the 'parallel computation' paradigm mentioned in the golden rule — spreading loop work across threads.

Q4. Write a program to calculate the sum of all elements of an array using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i, sum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};

    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < 10; i++) {
        sum = sum + arr[i];
    }

    printf("Total Sum = %d\n", sum);
    return 0;
}
```

Sample Output

```
Total Sum = 55
```

Explanation — Step by Step (Simple Words)

1. The array holds numbers 1 to 10. Expected sum = $1+2+\dots+10 = 55$, so this is easy to double-check.

2. `reduction(+:sum)` is the key part. It tells OpenMP: 'each thread should keep its OWN private mini-total of sum, add up its own share of the array, then at the end ADD all the mini-totals together into the final sum'.
3. Why not just make sum shared? Because if 4 threads try to update the same shared sum at the exact same time, some additions get LOST (this is called a race condition). `reduction` avoids this automatically and is also faster than using a lock every time.
4. The `+` in `reduction(+:sum)` is the operator being reduced. You could also see `*` (product), `max`, or `min` here.
5. After the loop finishes, `sum` already holds the fully combined, correct total — you just print it.

Note: This exact program is very likely to appear — memorize the `reduction(+:sum)` pattern well.

Q5. Modify the sum program to instead calculate the **PRODUCT** of all array elements.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int i;
    long product = 1;
    int arr[5] = {1,2,3,4,5};

    #pragma omp parallel for reduction(*:product)
    for (i = 0; i < 5; i++) {
        product = product * arr[i];
    }

    printf("Total Product = %ld\n", product);
    return 0;
}
```

Sample Output

```
Total Product = 120
```

Explanation — Step by Step (Simple Words)

1. Only two things changed from the sum example: the reduction operator is now `*` instead of `+`, and the loop body does multiplication instead of addition.
2. The starting value of `product` is 1 (not 0!) because multiplying by 0 would wipe everything out. For sums you start at 0, for products you start at 1 — this is the 'identity value' rule.
3. Each thread multiplies its own chunk of the array into its private copy of `product`, then OpenMP multiplies all the private copies together at the end.
4. $1 \times 2 \times 3 \times 4 \times 5 = 120$, matching the expected factorial-style result.

Note: This proves the golden rule: same skeleton as sum, only the operator and starting value change.

Q6. Write an OpenMP program to calculate the factorial of a number N.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int n = 6;
    long fact = 1;

    #pragma omp parallel for reduction(*:fact)
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }

    printf("Factorial of %d = %ld\n", n, fact);
    return 0;
}
```

Sample Output

```
Factorial of 6 = 720
```

Explanation — Step by Step (Simple Words)

1. Factorial of N means $N \times (N-1) \times (N-2) \times \dots \times 1$. Here $N=6$, so $6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$.
2. This is really the SAME pattern as Q5 (product reduction) — just looping from 1 to n instead of looping over array elements.
3. `reduction(*:fact)` again gives every thread a private safe copy of fact so multiplications from different threads don't clash.
4. fact starts at 1 (identity value for multiplication), matching the same rule as Q5.
5. If the teacher asks for factorial in the exam instead of sum, THIS is exactly the adaptation you make: change + to *, change the array loop to a 1..n loop.

Note: This is the teacher's exact example of a 'logic swap' — sum becomes factorial, structure stays identical.

Q7. Find the MAXIMUM value in an array using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int arr[8] = {12, 45, 3, 67, 21, 9, 88, 34};
    int max_val = arr[0];

    #pragma omp parallel for reduction(max:max_val)
    for (int i = 0; i < 8; i++) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }

    printf("Maximum value = %d\n", max_val);
    return 0;
}

```

Sample Output

```
Maximum value = 88
```

Explanation — Step by Step (Simple Words)

1. `max_val` is initialised to `arr[0]` so it already holds a valid starting value from the array.
2. `reduction(max:max_val)` gives each thread its own private 'best value found so far' from its own chunk of the array.
3. Inside the loop, each thread simply compares: 'is this element bigger than my current best? If yes, update it.'
4. At the end, OpenMP compares all the threads' private 'best' values against each other and keeps the overall largest one as the final `max_val`.
5. This is much safer than using a shared `max_val` directly, since two threads updating the same shared variable at once could cause one thread's update to be lost.

Note: Works the same way for `reduction(min:...)` — just flip the comparison to '`<`' as shown in Q8.

Q8. Find the MINIMUM value in an array using OpenMP.

Answer (Code)


```
#include <stdio.h>
#include <omp.h>

int main() {
    int arr[8] = {12, 45, 3, 67, 21, 9, 88, 34};
    int min_val = arr[0];

    #pragma omp parallel for reduction(min:min_val)
    for (int i = 0; i < 8; i++) {
        if (arr[i] < min_val) {
            min_val = arr[i];
        }
    }

    printf("Minimum value = %d\n", min_val);
    return 0;
}
```

Sample Output

```
Minimum value = 3
```

Explanation — Step by Step (Simple Words)

1. Exactly the mirror image of Q7 — everything is identical except the reduction word changes from max to min, and the comparison flips from '>' to '<'.
2. min_val starts at arr[0] as a safe starting guess.
3. Each thread keeps its own smallest-value-found-so-far privately.
4. OpenMP combines all threads' private smallest values and keeps the overall smallest as the answer.

Note: Q7 and Q8 together prove one more time: memorize ONE pattern, swap one word to answer both questions.

Q9. Count how many even numbers and how many odd numbers are in an array.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int evenCount = 0, oddCount = 0;

    #pragma omp parallel for reduction(+:evenCount, oddCount)
    for (int i = 0; i < 10; i++) {
        if (arr[i] % 2 == 0)
            evenCount++;
        else
            oddCount++;
    }

    printf("Even count = %d\n", evenCount);
    printf("Odd count = %d\n", oddCount);
    return 0;
}

```

Sample Output

```

Even count = 5
Odd count = 5

```

Explanation — Step by Step (Simple Words)

1. This shows you can reduce MORE THAN ONE variable at once — just separate them with a comma inside the same reduction(...) clause.
2. Each thread keeps its own private evenCount and oddCount, increasing whichever one matches the number it's currently checking (arr[i] % 2 == 0 means the number is even).
3. At the end, OpenMP adds together all threads' private evenCount values into one final evenCount, and does the same for oddCount.
4. Out of numbers 1 to 10, there are exactly 5 even (2,4,6,8,10) and 5 odd (1,3,5,7,9).

Note: A common exam twist: 'count elements above a value', 'count negatives', etc — same reduction(+:count) idea.

Q10. Show what goes WRONG if a shared variable is updated without any protection (race condition).

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int counter = 0;

    #pragma omp parallel for
    for (int i = 0; i < 100000; i++) {
        counter++;    // NOT safe - no reduction, no critical, no atomic
    }

    printf("Counter = %d (expected 100000)\n", counter);
    return 0;
}

```

Sample Output

```

Counter = 74213 (expected 100000)
-- the exact wrong number changes every run! --

```

Explanation — Step by Step (Simple Words)

1. counter is SHARED by default (it's declared outside the parallel region and not marked private).
2. counter++ is not really one instruction — it's actually 'read counter, add 1, write counter back'. This is called a **race condition**: two threads can both read the SAME old value at the same time, both add 1, and both write back the same new value — so one increment gets silently lost.
3. Because thread timing is unpredictable, the final wrong answer is DIFFERENT every time you run the program.
4. This is a classic 'explain what's wrong with this code' exam question. The fix is to add reduction(+:counter), or wrap counter++ in a critical or atomic section (see Q11 and Q12).

Note: If asked 'why is this output wrong / not always the same', THIS is the race-condition explanation to give.

Q11. Fix the race condition from Q10 using a critical section.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    int counter = 0;

    #pragma omp parallel for
    for (int i = 0; i < 100000; i++) {
        #pragma omp critical
        {
            counter++;
        }
    }

    printf("Counter = %d\n", counter);
    return 0;
}
```

Sample Output

```
Counter = 100000
```

Explanation — Step by Step (Simple Words)

1. `#pragma omp critical` marks the block right after it as a 'one thread at a time only' zone.
2. If one thread is currently inside the critical block, every other thread that reaches it must WAIT in line until the first one finishes.
3. This guarantees `counter++` always completes safely (read + add 1 + write back, with nobody interrupting), so the final answer is always exactly correct: 100000.
4. Downside: because threads must wait their turn, critical sections can slow the program down if used too often or on very large loops — reduction is usually faster when it applies.

Note: Use critical when the shared update is more complex than a simple `++`/`--`/`max`/`min` (things reduction can't handle directly).

Q12. Fix the same race condition using an atomic operation instead of critical.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int counter = 0;

    #pragma omp parallel for
    for (int i = 0; i < 100000; i++) {
        #pragma omp atomic
        counter++;
    }

    printf("Counter = %d\n", counter);
    return 0;
}

```

Sample Output

```
Counter = 100000
```

Explanation — Step by Step (Simple Words)

1. `#pragma omp atomic` also protects a shared update, but it only protects ONE simple statement right after it (no curly-brace block needed).
2. It works because the CPU itself can perform very simple operations like `++`, `--`, `+=`, `-=` as a single uninterruptible hardware step — so it's usually faster than `critical`.
3. Rule of thumb for exams: use `atomic` for a single simple line (`x++`, `x += value`); use `critical` for a multi-line block or anything more complex.
4. Result is the same as Q11: exactly 100000, with no lost updates.

Note: atomic vs critical is a common 'compare the two' theory question — atomic is lighter/faster but only for single simple statements.

Q13. Use OpenMP sections to run two DIFFERENT tasks on two different threads at the same time.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>

int main() {
    #pragma omp parallel sections
    {
        #pragma omp section
        {
            printf("Task A: computing sum...\n");
        }

        #pragma omp section
        {
            printf("Task B: computing average...\n");
        }
    }
    return 0;
}
```

Sample Output

```
Task A: computing sum...
Task B: computing average...
(order may vary)
```

Explanation — Step by Step (Simple Words)

1. So far every example split ONE loop's iterations across threads (data parallelism). sections is different: it hands completely DIFFERENT chunks of code to different threads (task parallelism).
2. `#pragma omp parallel sections` opens the region; each `#pragma omp section` block inside is one independent job.
3. OpenMP assigns each section block to a free thread. With 2 sections, typically 2 different threads run Task A and Task B simultaneously.
4. If there are more threads than sections, the extra threads simply have nothing to do and sit idle.

Note: Use sections when the teacher's question says 'do task X and task Y at the same time', not 'split this loop'.

Q14. Demonstrate the difference between a private and a shared variable in the same parallel region.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int shared_var = 100;
    int private_var = 100;

    #pragma omp parallel private(private_var) shared(shared_var)
    {
        int id = omp_get_thread_num();
        private_var = id * 10;        // each thread's own copy
        printf("Thread %d: private_var = %d, shared_var = %d\n",
              id, private_var, shared_var);
    }
    return 0;
}

```

Sample Output

```

Thread 0: private_var = 0,  shared_var = 100
Thread 1: private_var = 10, shared_var = 100
Thread 2: private_var = 20, shared_var = 100
Thread 3: private_var = 30, shared_var = 100

```

Explanation — Step by Step (Simple Words)

1. `private(private_var)` gives EVERY thread its own independent, uninitialised copy of `private_var` — that's why each thread can set it to a different value (`id * 10`) without affecting the others.
2. `shared(shared_var)` means all threads look at the exact SAME memory location — that's why every thread prints the same value, 100, for `shared_var`.
3. Important trap: a private variable's value BEFORE the parallel region is undefined inside it (it does not automatically keep the value 100) — you must set it again inside the region, which is exactly what `private_var = id * 10;` does here.
4. `shared` is actually the DEFAULT for variables declared outside the parallel region, so writing `shared(shared_var)` here is just for clarity — it would behave the same even without it.

Note: This is a very common 'explain the difference' theory question — expect it in written/short-answer form too.

Q15. Set the number of threads manually instead of using the system default.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    omp_set_num_threads(4);

    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int total = omp_get_num_threads();
        printf("Thread %d of %d\n", id, total);
    }
    return 0;
}

```

Sample Output

```

Thread 0 of 4
Thread 1 of 4
Thread 2 of 4
Thread 3 of 4

```

Explanation — Step by Step (Simple Words)

1. `omp_set_num_threads(4)` is called BEFORE the parallel region to force OpenMP to always use exactly 4 threads, no matter how many CPU cores the machine has.
2. `omp_get_num_threads( )` (called INSIDE the parallel region) returns how many threads are actually running in that region — here, 4.
3. You can also set the number of threads directly on the pragma line instead: `#pragma omp parallel num_threads(4)` — same effect, without needing the separate function call.
4. Without either of these, OpenMP picks a default number of threads, usually equal to the number of CPU cores available.

Note: Two ways to fix thread count: `omp_set_num_threads(n)` before the region, OR `num_threads(n)` on the pragma itself.

Q16. Add two matrices (2D arrays) together using OpenMP.

Answer (Code)


```

#include <stdio.h>
#include <omp.h>

#define N 3

int main() {
    int A[N][N] = {{1,2,3},{4,5,6},{7,8,9}};
    int B[N][N] = {{9,8,7},{6,5,4},{3,2,1}};
    int C[N][N];

    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

Sample Output

```

10 10 10
10 10 10
10 10 10

```

Explanation — Step by Step (Simple Words)

1. `#pragma omp parallel for` is placed on the OUTER loop (over rows `i`) only — this splits the ROWS of the matrix among threads.
2. Each thread that gets a row `i` fully computes that entire row using a normal, ordinary inner loop over `j` — no pragma is needed on the inner loop.
3. This works safely with no reduction/critical needed because every thread writes to a DIFFERENT row of `C` — there is no overlap, so no two threads ever touch the same memory cell.
4. Every `A[i][j] + B[i][j]` here happens to equal 10, which is why the whole result matrix is 10s — an easy way to sanity-check your answer.

Note: General rule for nested loops: put the pragma on whichever loop has independent, non-overlapping work per iteration (usually the outer one).

Q17. Calculate the sum of squares of numbers from 1 to N using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int n = 5;
    long sumSquares = 0;

    #pragma omp parallel for reduction(+:sumSquares)
    for (int i = 1; i <= n; i++) {
        sumSquares += i * i;
    }

    printf("Sum of squares 1..%d = %ld\n", n, sumSquares);
    return 0;
}

```

Sample Output

```
Sum of squares 1..5 = 55
```

Explanation — Step by Step (Simple Words)

1. $1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1+4+9+16+25 = 55$ — matches the printed result.
2. This is the exact same sum-reduction skeleton as Q4, just with $i*i$ inside the loop instead of $arr[i]$, and looping directly over numbers $1..n$ instead of reading from an array.
3. `reduction(+:sumSquares)` again keeps a safe private running total per thread, then adds them all together at the end.
4. This confirms the golden rule one more time: same `#pragma`, only the math inside the loop body changes.

Note: Any 'sum of ... from 1 to N' question (sum of cubes, sum of odd numbers, etc.) follows this identical pattern.

Q18. Search for a target value inside an array in parallel and report if/where it was found.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>

int main() {
    int arr[8] = {5, 12, 9, 33, 47, 6, 21, 8};
    int target = 47;
    int foundIndex = -1;

    #pragma omp parallel for
    for (int i = 0; i < 8; i++) {
        if (arr[i] == target) {
            #pragma omp critical
            {
                foundIndex = i;
            }
        }
    }

    if (foundIndex != -1)
        printf("Found %d at index %d\n", target, foundIndex);
    else
        printf("%d not found\n", target);

    return 0;
}

```

Sample Output

```
Found 47 at index 4
```

Explanation — Step by Step (Simple Words)

1. The parallel for still splits the 8 array positions across the available threads, so different threads check different elements at the same time.
2. foundIndex is a SHARED variable (declared outside the region), so whichever thread finds a match must write to it safely — that's why the write is wrapped in #pragma omp critical.
3. reduction is not used here because 'store the index where I found it' is not a simple sum/product/max/min operation — critical is the right tool for this kind of custom shared update.
4. If the array had duplicate target values, whichever thread happens to write LAST would 'win' — for a search that should return the FIRST match, extra care (like reduction(min:foundIndex) on valid indices only) would be needed, but the simple version above is what's normally expected.

Note: This shows the two paradigms working together: parallel for (splitting work) + critical (safe shared write) in one program.

Quick Revision Cheat-Sheet

#pragma omp parallel	Start a team of threads; every thread runs the block.
#pragma omp parallel for	Start threads AND split a for-loop's iterations among them.
private(var)	Each thread gets its own separate copy of var.

shared(var)	All threads use the exact same var (default for outer variables).
reduction(+:var)	Safely combine a variable across threads with +. Also works with *, max, min.
critical	Only ONE thread at a time may run this block — others wait in line.
atomic	Protects a single simple statement (x++, x+=v) very cheaply.
sections / section	Give different, independent chunks of code to different threads.
omp_get_thread_num()	Returns the ID (0,1,2,...) of the calling thread.
omp_get_num_threads()	Returns how many threads exist in the current region.
omp_set_num_threads(n)	Force OpenMP to use exactly n threads.
num_threads(n)	Same as above, written directly on the pragma line.

Exam strategy: read the question first, identify WHICH of the two paradigms it needs (split a loop vs. protect shared memory), then reuse the closest matching skeleton above. You do not need to memorize 18 different programs — you need to memorize ONE skeleton per paradigm and know how to slot the new logic in.